

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

APLICACIONES DISTRIBUIDAS (ISR-701)

GA-SUM-03 / PE-U3

Implementación y Verificación de un Clúster de Base de Datos Distribuida con CockroachDB

aplicado al PFC

FUVV – Laboratorios Informáticos

Reserva y monitoreo distribuido de laboratorios

Equipo:	Equipo B
Unidad:	Unidad 3 – Bases de Datos Distribuidas
Modalidad:	Grupal
Docente:	Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico:	2026–2027 PPA

Integrantes y PFC de origen

N.º	Integrante	PFC de origen
1	[Lozano Morales José Alejandro]	[AGLS]
2	[Sanchez Pilaloe Andy Paul]	[AGLS]
3	[Urbina Isaias Romero Abraham]	[FUVV]

Justificación de la selección del PFC. El dominio FUVV resulta apropiado para implementar una base de datos distribuida porque las reservas, los laboratorios, los equipos y los eventos de monitoreo generan información relacionada que puede crecer y consultarse desde distintas ubicaciones. La distribución por laboratorio o zona permite experimentar con fragmentación horizontal, mientras que la replicación protege la disponibilidad de reservas y estados operativos. Además, la concurrencia entre solicitudes simultáneas exige transacciones consistentes y ofrece un escenario claro para verificar tolerancia a fallos y recuperación mediante CockroachDB.

Índice

1. Introducción	1
1.1. Objetivos	2
1.2. Organización del informe	2
2. Fundamento teórico	3
2.1. Fundamentos y arquitecturas de bases de datos distribuidas	3
2.2. Fragmentación y replicación de datos	4
2.3. Control de concurrencia distribuido y consenso Raft	5
2.4. Consistencia, disponibilidad, PACELC y sistemas NewSQL/NoSQL	7
3. Diseño aplicado al PFC FUVV	8
3.1. Organización del proyecto	8
3.2. Esquema relacional del dominio	9
3.3. Estrategia de fragmentación y replicación	10
4. Procedimiento experimental	11
4.1. Despliegue del clúster	11
4.2. Creación del esquema	13
4.3. Aplicación y verificación del particionamiento	14
4.4. Consultas de validación	15
4.5. Carga masiva y validación de datos	16
4.6. Ejecución del benchmark del clúster	17
5. Resultados iniciales	18
5.1. Análisis de los planes de ejecución	19
5.2. Prueba de tolerancia a fallos y reintegración	24
6. Comparación de rendimiento con nodo único	28
7. Preguntas de análisis	29
7.1. ¿Cómo garantiza Raft la consistencia cuando falla un nodo?	29
7.2. CockroachDB frente a Cassandra desde la consistencia y PACELC	30
7.3. Fragmentación horizontal explícita para la tabla principal del PFC	31
7.4. Atomicidad de transacciones multitabla y relación entre commit y Raft	32
8. Conclusiones individuales	32
8.1. Conclusión de José Alejandro Lozano Morales	32

9. Declaración de uso de inteligencia artificial generativa	33
9.1. Conclusión de Andy Paul Sanchez Pilaloa	33
Bibliografía	34

Resumen

Este informe presenta el diseño y la implementación experimental de un clúster de base de datos distribuida con CockroachDB aplicado al PFC FUVV – Laboratorios Informáticos. La solución organiza la información de usuarios, laboratorios, equipos, asignaciones, asistencias y reportes de fallos en un esquema relacional de diez tablas. El entorno fue desplegado mediante Docker Compose sobre tres nodos, con volúmenes persistentes, localidades lógicas y replicación de los datos. Como estrategia de distribución se aplicó particionamiento horizontal por lista sobre el estado de los reportes de fallo, con preferencias de leaseholder diferenciadas para cada partición. Se generó un conjunto de 15 330 registros sintéticos y se ejecutaron cinco consultas representativas con diez repeticiones por punto de acceso. La prueba de fallos confirmó la continuidad de lectura al detener un nodo, mientras que la comparación experimental mostró un promedio global de 8.3980 ms en el clúster y 9.5750 ms en la instancia de nodo único. El clúster obtuvo menor latencia en tres de las cinco consultas, aunque el nodo único fue más rápido en las otras dos, lo que demuestra que la distribución no produce una mejora uniforme y que su efecto depende del plan de ejecución y del costo de coordinación.

Palabras clave: base de datos distribuida, CockroachDB, fragmentación horizontal, replicación, consenso Raft, laboratorios informáticos.

1. Introducción

Los sistemas de información que atienden varias ubicaciones deben conservar una visión lógica coherente de los datos aun cuando el almacenamiento, el procesamiento y las comunicaciones se distribuyan entre distintos nodos. Una base de datos distribuida persigue precisamente esa integración: ofrece a las aplicaciones una interfaz común mientras coordina fragmentos, réplicas, transacciones y fallos en una infraestructura descentralizada. Este enfoque puede mejorar escalabilidad, proximidad de acceso y continuidad operativa, pero introduce costos de coordinación que no aparecen con la misma intensidad en una instancia centralizada [1], [2].

El PFC FUVV – Laboratorios Informáticos comprende la reserva, administración y supervisión de laboratorios y equipos. En este dominio pueden coincidir solicitudes de uso, registros de asistencia y reportes de fallos generados desde diferentes espacios físicos. Una solución distribuida debe impedir, por ejemplo, que dos operaciones concurrentes produzcan estados incompatibles para una reserva o que un fallo de infraestructura vuelva inaccesible toda la información operativa. Por ello, el dominio permite estudiar de manera concreta la fragmentación, la replicación, la consistencia transaccional y la tolerancia a fallos.

CockroachDB fue seleccionado como plataforma experimental porque combina una interfaz SQL relacional con replicación por consenso, transacciones distribuidas y escalabilidad horizontal. Su arquitectura divide el espacio de claves en rangos replicados

y coordina las modificaciones mediante grupos de consenso, mientras ofrece aislamiento serializable a las aplicaciones. Estas propiedades buscan mantener la corrección de las transacciones frente a fallos de nodos y particiones de red, aunque la coordinación puede aumentar la latencia de ciertas operaciones [3], [4].

La práctica se organiza como un experimento reproducible. Primero se diseña el esquema de datos de FUVV y la estrategia de particionamiento; después se despliega un clúster de tres nodos, se cargan datos sintéticos y se verifican las configuraciones de distribución. Finalmente se ejecutan consultas representativas tanto en el clúster como en una instancia independiente de nodo único, se conservan las mediciones brutas y se relacionan los tiempos observados con los planes de ejecución. Esta comparación evita atribuir a la distribución una ventaja automática y permite reconocer los casos en los que el costo de coordinación supera los beneficios del procesamiento distribuido.

1.1. Objetivos

Objetivo general. Implementar, configurar y verificar un clúster funcional de base de datos distribuida de tres nodos aplicado al PFC FUVV, demostrando la distribución y replicación de los datos, su comportamiento ante fallos y el rendimiento de consultas representativas.

Objetivos específicos.

1. Configurar tres nodos CockroachDB en contenedores Docker con red y almacenamiento persistente.
2. Implementar el esquema relacional de laboratorios informáticos y una estrategia explícita de fragmentación horizontal.
3. Verificar la carga, distribución y replicación de los datos mediante consultas SQL y evidencia del clúster.
4. Comprobar experimentalmente la continuidad del servicio ante la caída y posterior reintegración de un nodo.
5. Comparar cinco consultas del PFC en el clúster y en una instancia de nodo único, relacionando los tiempos con sus planes de ejecución.

1.2. Organización del informe

Después de esta introducción se presenta el fundamento teórico sobre arquitecturas distribuidas, fragmentación, replicación, concurrencia, consenso y modelos de consistencia. A continuación se describe el diseño aplicado a FUVV, el procedimiento experimental, los resultados del clúster, la comparación con nodo único y la prueba de tolerancia a fallos. Las secciones finales desarrollan las preguntas obligatorias de análisis, las conclusiones

individuales, la declaración de uso de inteligencia artificial y la bibliografía en formato IEEE.

2. Fundamento teórico

2.1. Fundamentos y arquitecturas de bases de datos distribuidas

Una base de datos distribuida es una colección lógicamente integrada de datos que se almacena físicamente en varios sitios conectados por una red. El sistema de gestión de bases de datos distribuidas añade los mecanismos necesarios para presentar esa colección como una sola base: localiza los datos, transforma consultas globales en operaciones sobre fragmentos, coordina transacciones y mantiene la coherencia entre réplicas. La distribución no equivale simplemente a permitir conexiones remotas a un servidor central; requiere que parte del almacenamiento o del procesamiento resida en nodos autónomos y que el sistema oculte, hasta donde resulte posible, la complejidad de su coordinación [1], [2].

Los objetivos atribuidos a Date constituyen una referencia para evaluar esa transparencia. Comprenden autonomía local; ausencia de dependencia de un sitio central; operación continua; independencia de ubicación; independencia de fragmentación; independencia de replicación; procesamiento distribuido de consultas; gestión distribuida de transacciones; independencia del hardware; independencia del sistema operativo; independencia de la red; e independencia del SGBD. No todos se alcanzan de forma absoluta en una plataforma concreta, pero sirven para separar la visión lógica ofrecida al usuario de las decisiones físicas de despliegue. En FUVV, por ejemplo, una consulta de reportes no debería exigir que el usuario conozca qué nodo mantiene el leaseholder del rango ni cuántas réplicas intervienen en la lectura [1], [3].

Las arquitecturas distribuidas pueden organizarse de diferentes maneras. En una arquitectura cliente-servidor, los clientes formulan solicitudes y uno o varios servidores especializados almacenan y procesan datos. En una arquitectura colaborativa o entre pares, los nodos pueden actuar tanto como solicitantes como proveedores y participan en la ejecución global. Una capa de middleware, por su parte, proporciona servicios comunes sobre fuentes que pueden haber sido diseñadas de forma independiente: traducción de consultas, catálogo global, seguridad o coordinación transaccional. CockroachDB adopta una arquitectura distribuida homogénea en la que todos los nodos ejecutan el mismo software y pueden recibir tráfico SQL, mientras la capa interna enruta las operaciones hacia los rangos correspondientes [3], [4].

La transparencia puede analizarse en ocho dimensiones complementarias. La transparencia de fragmentación oculta la división de una relación; la de replicación evita que la aplicación administre copias; la de ubicación separa los nombres lógicos de los sitios físicos; y la de acceso ofrece operaciones uniformes para datos locales y remotos. La transparencia de concurrencia busca que las transacciones simultáneas produzcan resultados equivalentes a una ejecución correcta; la de fallos preserva propiedades definidas cuando ciertos

componentes dejan de responder; la de escalabilidad permite aumentar nodos o carga sin rediseñar por completo la aplicación; y la de evolución facilita cambios del esquema o la infraestructura. Estas transparencias tienen límites: la latencia y la disponibilidad observables pueden revelar la distribución, aunque la interfaz lógica permanezca estable [2], [5].

Finalmente, una BDD homogénea emplea modelos de datos, protocolos y SGBD compatibles entre los sitios, lo que simplifica catálogo, optimización y transacciones. Una BDD heterogénea integra plataformas o esquemas diferentes y necesita resolver incompatibilidades sintácticas, semánticas y operativas. El clúster de esta práctica es homogéneo porque los tres nodos ejecutan la misma versión de CockroachDB y comparten un único esquema. Sin embargo, el análisis de heterogeneidad sigue siendo relevante para una evolución futura de FUVV que integre sensores, inventarios externos o servicios institucionales mediante middleware [1], [6].

2.2. Fragmentación y replicación de datos

La fragmentación divide una relación global en subconjuntos que pueden almacenarse o procesarse de forma separada. Su diseño debe cumplir condiciones de corrección para que la distribución no altere el significado de la base. Sea una relación R y un conjunto de predicados $p_1, \dots, p_n$. La fragmentación horizontal primaria produce cada fragmento mediante selección:

$$R_i = \sigma_{p_i}(R), \quad 1 \leq i \leq n. \quad (1)$$

La condición de completitud exige que cada tupla de la relación global aparezca en al menos un fragmento. Si los predicados cubren el dominio de la clave de fragmentación, se expresa como:

$$\bigcup_{i=1}^n R_i = R. \quad (2)$$

La reconstrucción establece que debe existir una operación relacional capaz de recuperar R a partir de sus fragmentos. Para una fragmentación horizontal completa, esa operación es la unión:

$$R = R_1 \cup R_2 \cup \dots \cup R_n. \quad (3)$$

Cuando se exige disjunción, una tupla no debe pertenecer simultáneamente a dos fragmentos distintos:

$$R_i \cap R_j = \emptyset, \quad \forall i \neq j. \quad (4)$$

Estas propiedades evitan pérdida, duplicación no intencional y resultados ambiguos

durante la reconstrucción. En FUVV, los predicados `estado='PENDIENTE'`, `estado='EN_PROCESO'` y `estado='RESUELTO'` son mutuamente excluyentes; serán completos siempre que el dominio permitido de `estado` se limite a esos tres valores [1], [7].

La fragmentación horizontal derivada distribuye una relación basándose en la fragmentación de otra con la que mantiene una relación semántica. Si $S_i = \sigma_{p_i}(S)$ son fragmentos de S y R contiene una clave foránea que referencia a S , puede definirse:

$$R_i = R \bowtie_{R.k=S.k} S_i. \quad (5)$$

La semijunta conserva en R_i las tuplas relacionadas con el fragmento S_i . Aplicado a FUVV, si `laboratorio` se fragmentara por zona, los equipos podrían fragmentarse de manera derivada según el laboratorio al que pertenecen. Esta colocación favorece uniones locales, aunque debe conservarse la completitud de las relaciones y tratar explícitamente los casos sin correspondencia [1], [8].

En la fragmentación vertical se agrupan atributos en lugar de tuplas. Cada fragmento debe conservar una clave o identificador de tupla K para permitir la reconstrucción. Si A_i es un subconjunto de atributos, se define $R_i = \pi_{K \cup A_i}(R)$ y la relación global se recupera mediante:

$$R = R_1 \bowtie_K R_2 \bowtie_K \cdots \bowtie_K R_n. \quad (6)$$

La afinidad de atributos estima qué columnas se consultan juntas para evitar uniones frecuentes entre fragmentos. La fragmentación mixta aplica de forma sucesiva operaciones horizontales y verticales; ofrece mayor flexibilidad, pero también incrementa la complejidad del catálogo, la optimización y la reconstrucción [1], [6].

La replicación mantiene copias de un fragmento en varios nodos. En un esquema síncrono, una escritura se confirma cuando alcanza el criterio de acuerdo definido, lo que reduce la posibilidad de lecturas obsoletas a costa de latencia y dependencia del quórum. En uno asíncrono, el nodo principal puede responder antes de que todas las copias reciban el cambio; esto favorece la latencia y disponibilidad local, pero abre una ventana de inconsistencia. CockroachDB replica rangos y utiliza consenso para ordenar las modificaciones: la confirmación depende de una mayoría, no de que todos los nodos respondan. En la práctica de FUVV se configuraron tres réplicas, una por zona lógica, y preferencias de leaseholder distintas para las particiones. De este modo, la partición orienta la ubicación preferida del tráfico, mientras la replicación preserva copias para mantener disponibilidad ante una falla minoritaria [3], [4].

2.3. Control de concurrencia distribuido y consenso Raft

El control de concurrencia debe impedir que transacciones simultáneas violen las reglas de integridad o expongan estados intermedios. En una base distribuida el problema se amplía porque los datos y las decisiones pueden residir en nodos diferentes, los

mensajes sufren demoras y un participante puede fallar mientras otros continúan activos. La serializabilidad establece que el efecto de una ejecución concurrente debe ser equivalente al de algún orden secuencial de las mismas transacciones. Para FUVV esta propiedad es relevante cuando varias solicitudes intentan reservar el mismo laboratorio o cuando un reporte cambia de estado mientras otro proceso consulta la disponibilidad del equipo [1], [2].

El bloqueo en dos fases (2PL) separa la vida de una transacción en una fase de crecimiento, durante la cual adquiere bloqueos, y otra de decrecimiento, en la que los libera sin obtener nuevos. El 2PL estricto conserva los bloqueos de escritura hasta confirmar o abortar, evitando que otras transacciones observen cambios no comprometidos y simplificando la recuperación. En un entorno distribuido, los gestores de bloqueo deben coordinar recursos ubicados en varios sitios. Esto puede generar interbloqueos globales que no son visibles desde un único nodo. Un grafo *wait-for* representa una arista $T_i \rightarrow T_j$ cuando T_i espera un recurso retenido por T_j ; un ciclo indica un deadlock y obliga a seleccionar una transacción víctima [1], [5].

La atomicidad entre participantes suele explicarse mediante el protocolo de commit en dos fases (2PC). En la fase de preparación, el coordinador solicita a cada participante que vote después de asegurar localmente que puede confirmar. Si todos votan afirmativamente, en la segunda fase se comunica la decisión de commit; cualquier voto negativo conduce al aborto. El protocolo conserva una decisión uniforme, pero puede bloquear participantes preparados si el coordinador falla antes de comunicar la resolución. El 3PC incorpora una fase intermedia y supuestos temporales para reducir ciertos escenarios de bloqueo, aunque aumenta mensajes y no elimina todos los problemas en redes asíncronas. Los sistemas modernos también emplean optimizaciones, registros replicados o flujos con estado para disminuir el costo de coordinación[9], [10].

El consenso resuelve un problema diferente pero relacionado: conseguir que réplicas no defectuosas acuerden un orden de operaciones. Raft organiza cada grupo en seguidores, candidatos y un líder. Los seguidores esperan mensajes periódicos; si expira el temporizador de elección, un nodo pasa a candidato, incrementa el término y solicita votos. Un candidato que obtiene mayoría se convierte en líder. El líder recibe comandos, los incorpora a su log y los replica; una entrada se considera comprometida cuando una mayoría la ha almacenado bajo las reglas del protocolo. Los términos, la restricción de voto y la coincidencia de prefijos impiden que dos historiales incompatibles se consideren simultáneamente comprometidos[3], [11].

CockroachDB divide los datos en rangos y asocia cada rango con un grupo de réplicas coordinado por consenso. El leaseholder atiende gran parte de las solicitudes y dirige propuestas hacia el grupo Raft, pero la durabilidad de una escritura no depende de una sola máquina. Con tres réplicas, la mayoría es dos; por ello, el sistema puede continuar confirmando operaciones si un nodo falla, siempre que las dos réplicas restantes puedan comunicarse. Si se pierde la mayoría, se prioriza la seguridad y el rango deja de aceptar operaciones que no puedan acordarse. Esta relación entre quórum, orden del log y

transacciones serializables constituye la base de la prueba de tolerancia a fallos pendiente en FUVV [3], [4].

2.4. Consistencia, disponibilidad, PACELC y sistemas NewSQL/NoSQL

El teorema CAP formaliza una limitación de los sistemas de datos replicados: cuando existe una partición de red que impide la comunicación entre grupos de nodos, no es posible garantizar simultáneamente consistencia fuerte y disponibilidad para todas las solicitudes. En este contexto, consistencia significa que las operaciones respetan una visión única compatible con el orden definido por el sistema; disponibilidad exige que toda solicitud a un nodo no fallido reciba una respuesta, aunque esta no pueda incorporar los cambios del otro lado de la partición. La tolerancia a particiones no es una preferencia que pueda eliminarse cuando la red realmente se divide: el diseño debe decidir qué garantía sacrifica durante ese evento [12], [13].

CAP describe el escenario de partición, pero no caracteriza el comportamiento durante la operación normal. El modelo PACELC amplía el análisis: si ocurre una partición (P), se elige entre disponibilidad (A) y consistencia (C); en ausencia de partición (E, *else*), persiste una elección entre latencia (L) y consistencia (C). Esta formulación muestra que incluso una red saludable paga el costo de coordinar réplicas cuando exige garantías fuertes. Los mecanismos basados en quórum permiten variar el número de réplicas requeridas para leer y escribir, modificando el equilibrio entre datos recientes, latencia y probabilidad de respuesta **noudoust2022quorum**, [14].

Los modelos de consistencia forman un espectro. La consistencia linealizable ordena cada operación como si ocurriera en un instante entre su invocación y respuesta; la serializabilidad ordena transacciones completas de manera equivalente a una ejecución secuencial. La consistencia causal preserva relaciones de causa y efecto, mientras la consistencia eventual permite divergencia temporal con la expectativa de convergencia si cesan las actualizaciones. No son etiquetas intercambiables: una base puede ofrecer serializabilidad transaccional y, además, garantías específicas para lecturas en tiempo real. La elección debe relacionarse con las anomalías que la aplicación puede tolerar [2], [13].

Los sistemas NoSQL suelen clasificarse por su modelo de datos. Los almacenes clave-valor priorizan acceso por clave y escalabilidad; Dynamo constituye una referencia para replicación y disponibilidad con reconciliación de versiones. Las bases documentales, como MongoDB, almacenan estructuras flexibles; las de columnas anchas, como Cassandra, distribuyen filas según claves de partición y ofrecen niveles de consistencia configurables; y las bases de grafos optimizan recorridos entre entidades relacionadas. Spanner y los sistemas NewSQL siguen otra dirección: conservan un modelo relacional y transacciones fuertes mientras distribuyen almacenamiento y procesamiento. Spanner utiliza tiempo coordinado y replicación para transacciones globales, mientras CockroachDB adopta rangos, consenso y una interfaz compatible con SQL [15], [16].

En una comparación cualitativa, CockroachDB prioriza consistencia serializable y continuidad mientras exista quórum; durante una partición que impide formar mayoría, rechaza operaciones antes que confirmar historiales incompatibles. Cassandra suele favorecer disponibilidad y rendimiento distribuido mediante consistencia ajustable, por lo que ciertas configuraciones aceptan lecturas temporalmente obsoletas. MongoDB emplea una arquitectura de primario y secundarios con opciones de preocupación de lectura y escritura; su posición no debe reducirse a una etiqueta única porque depende de la configuración. En los tres casos, topología, factor de replicación y política de confirmación afectan el equilibrio entre consistencia, disponibilidad y latencia [14], [17].

Para FUVV se prefiere CockroachDB porque una reserva duplicada, una asignación contradictoria o un estado de equipo perdido representan errores funcionales, no simples retrasos de visualización. La consistencia fuerte facilita imponer restricciones y transacciones sobre varias tablas, mientras el quórum de tres réplicas permite tolerar la caída de un nodo. El costo esperado es una mayor coordinación frente a alternativas eventualmente consistentes; por eso el benchmark debe medir latencia sin confundir velocidad con corrección. Esta elección corresponde a una postura PC durante particiones y EC en la rama normal de PACELC: se prioriza consistencia aun cuando ello implique coordinación adicional **noudoust2022quorum**, [3].

3. Diseño aplicado al PFC FUVV

La solución experimental se aplicó al PFC FUVV – Laboratorios Informáticos, cuyo dominio comprende la administración de laboratorios, equipos, usuarios, asignaciones, asistencias y reportes de fallos. El contexto de este proyecto resulta adecuado para una base de datos distribuida porque combina información administrativa con eventos operativos que pueden originarse simultáneamente desde distintos laboratorios. El diseño busca conservar las relaciones propias de un modelo SQL y, al mismo tiempo, distribuir el acceso a los reportes de monitoreo. Esta decisión concuerda con los principios de diseño de bases distribuidas, en los que fragmentación, asignación y replicación se estudian como decisiones relacionadas y no como operaciones aisladas [1], [6].

3.1. Organización del proyecto

El repositorio separa la definición del clúster, las sentencias SQL, los programas de carga y medición, la documentación y la evidencia. La Figura 1 muestra esta organización. El archivo `docker-compose.yml` contiene la topología de tres nodos; el directorio `sql/` conserva el esquema, la estrategia de particionamiento y las consultas; `scripts/` incluye la carga sintética y el benchmark; y `evidencia/` almacena las capturas y los resultados brutos. Esta separación permite reproducir cada fase y relacionar el resultado observado con el artefacto que lo produjo.

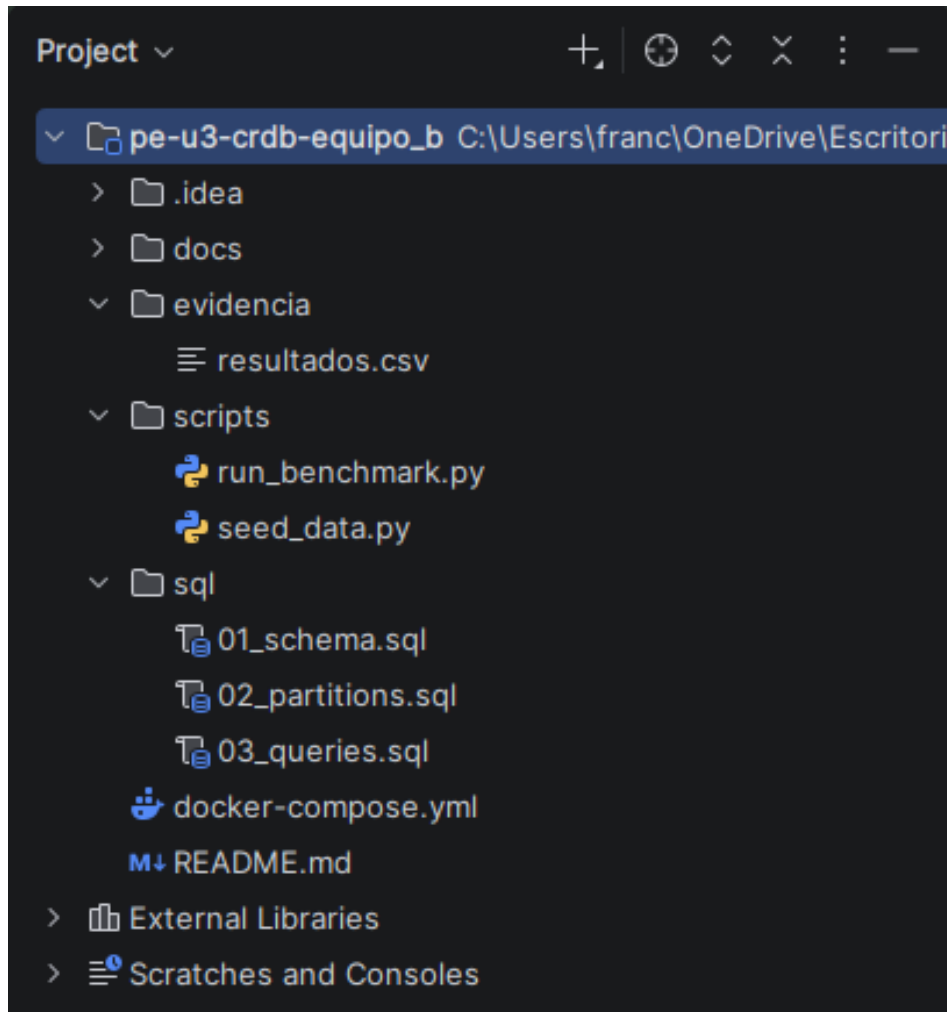


Figura 1: Estructura de archivos utilizada para la implementación del clúster.

3.2. Esquema relacional del dominio

La base de datos `gestion_laboratorios` está compuesta por diez tablas relacionadas. La Tabla 1 resume su responsabilidad dentro del PFC. La separación entre entidades personales, recursos, actividades y eventos de monitoreo permite conservar integridad referencial mediante claves primarias y foráneas, y proporciona varias rutas de consulta representativas para evaluar el clúster.

Cuadro 1: Tablas principales del esquema aplicado al PFC FUVV.

Tabla	Responsabilidad en el dominio
persona	Datos comunes de los usuarios registrados.
docente y estudiante	Especialización de las personas que utilizan los laboratorios.
laboratorio	Identificación, ubicación y características de cada laboratorio.
equipo	Inventario de equipos asociados con un laboratorio.
materia	Asignaturas vinculadas con el uso académico de los recursos.
asignacion_laboratorio	Programación del uso de laboratorios por materia y docente.
registro_asistencia	Cabecera de una sesión de asistencia.
detalle_asistencia	Participación individual de estudiantes en cada sesión.
reporte_fallo	Eventos de monitoreo y seguimiento de fallos de los equipos.

La tabla `reporte_fallo` fue seleccionada como relación principal para el ejercicio de distribución. La elección se justifica porque sus filas representan eventos operativos que cambian de estado durante su ciclo de vida y se consultan frecuentemente junto con `equipo` y `laboratorio`. En consecuencia, permite estudiar tanto filtros puntuales como uniones y agregaciones vinculadas directamente con el monitoreo del PFC.

3.3. Estrategia de fragmentación y replicación

La implementación utiliza particionamiento horizontal por lista sobre el atributo `estado`. Se definieron las particiones `reporte_pendiente`, `reporte_en_proceso` y `reporte_resuelto`. Antes de particionar, la clave primaria se transformó en la clave compuesta (`estado`, `id_reporte`) para que el atributo de partición fuera prefijo de la clave. El Listado 1 presenta las sentencias principales.

```

1 ALTER TABLE reporte_fallo
2   ADD CONSTRAINT uq_reporte_fallo_id
3   UNIQUE (id_reporte);
4
5 ALTER TABLE reporte_fallo ALTER PRIMARY KEY USING COLUMNS (estado,
6   id_reporte);
7
8 ALTER TABLE reporte_fallo PARTITION BY LIST (estado) ( PARTITION
9   reporte_pendiente VALUES IN ('PENDIENTE'), PARTITION
10  reporte_en_proceso VALUES IN ('EN_PROCESO'), PARTITION
11  reporte_resuelto VALUES IN ('RESUELTO') );

```

```

8
9 ALTER PARTITION reporte_pendiente OF TABLE reporte_fallo CONFIGURE ZONE
  USING num_replicas = 3, constraints = '{"+zone=nodo1": 1, "+zone=
    nodo2": 1, "+zone=nodo3": 1}', lease_preferences = '[[+zone=nodo1]]';
10
11 ALTER PARTITION reporte_en_proceso OF TABLE reporte_fallo CONFIGURE ZONE
  USING num_replicas = 3, constraints = '{"+zone=nodo1": 1, "+zone=
    nodo2": 1, "+zone=nodo3": 1}', lease_preferences = '[[+zone=nodo2]]';
12
13 ALTER PARTITION reporte_resuelto
14 OF TABLE reporte_fallo
15 CONFIGURE ZONE USING
16   num_replicas = 3,
17   constraints = '{"+zone=nodo1": 1,
18                 "+zone=nodo2": 1,
19                 "+zone=nodo3": 1}',
20   lease_preferences = '[[+zone=nodo3]]';

```

Listing 1: Fragmentación horizontal por lista y configuración de réplicas.

Cada partición conserva tres réplicas, con una restricción de ubicación para cada zona lógica: **nodo1**, **nodo2** y **nodo3**. La preferencia de leaseholder asigna prioritariamente los reportes pendientes al nodo 1, los reportes en proceso al nodo 2 y los resueltos al nodo 3. Por tanto, la estrategia no mantiene una única copia exclusiva en un nodo: distribuye la preferencia de atención mientras conserva tres copias para disponibilidad. Este comportamiento diferencia la ubicación preferida del leaseholder de la replicación física de los rangos, distinción importante en la arquitectura de CockroachDB [3], [4].

La selección por estado beneficia consultas operativas como la bandeja de fallos pendientes o el seguimiento de incidencias en proceso. No obstante, el estado es mutable; por ello, cuando una incidencia cambia de estado puede ser necesario trasladar su clave al nuevo espacio particionado. Este costo deberá considerarse posteriormente en el análisis crítico. La literatura confirma que una estrategia de fragmentación debe evaluarse conjuntamente con el patrón real de acceso y el costo de redistribución [7], [8].

4. Procedimiento experimental

4.1. Despliegue del clúster

El clúster se definió con tres servicios CockroachDB versión 25.2.3 y un servicio auxiliar de inicialización. Los servicios comparten una red bridge, pero mantienen volúmenes persistentes independientes. Los puertos SQL externos 26257, 26258 y 26259 permiten conectarse a cada nodo; las interfaces web se publican en 8080, 8081 y 8082. El parámetro `-join` proporciona la lista de nodos participantes y `-locality` asigna una zona distinta a cada contenedor. La configuración completa se presenta en el Listado 2.

```

1 services:

```

```

2   crdb-node1:
3     image: cockroachdb/cockroach:v25.2.3
4     container_name: crdb-node1
5     hostname: crdb-node1
6     command:
7       - start
8       - --insecure
9       - --advertise-addr=crdb-node1:26357
10      - --listen-addr=crdb-node1:26357
11      - --sql-addr=crdb-node1:26257
12      - --http-addr=crdb-node1:8080
13      - --join=crdb-node1:26357,crdb-node2:26357,crdb-node3:26357
14      - --locality=region=uteq,zone=nodo1
15      - --store=/cockroach/cockroach-data
16     ports:
17       - "26257:26257"
18       - "8080:8080"
19     volumes:
20       - crdb-node1-data:/cockroach/cockroach-data
21     networks:
22       - crdb-network
23     restart: unless-stopped
24
25 crdb-node2: image: cockroachdb/cockroach:v25.2.3 container_name: crdb-
26 node2 hostname: crdb-node2 command: - start - --insecure - --
27 advertise-addr=crdb-node2:26357 - --listen-addr=crdb-node2:26357 - --
28 sql-addr=crdb-node2:26257 - --http-addr=crdb-node2:8080 - --join=crdb-
29 node1:26357,crdb-node2:26357,crdb-node3:26357 - --locality=region=
30 uteq,zone=nodo2 - --store=/cockroach/cockroach-data ports: - "
31 26258:26257" - "8081:8080" volumes: - crdb-node2-data:/cockroach/
32 cockroach-data networks: - crdb-network restart: unless-stopped
33
34 crdb-node3: image: cockroachdb/cockroach:v25.2.3 container_name: crdb-
35 node3 hostname: crdb-node3 command: - start - --insecure - --
36 advertise-addr=crdb-node3:26357 - --listen-addr=crdb-node3:26357 - --
37 sql-addr=crdb-node3:26257 - --http-addr=crdb-node3:8080 - --join=crdb-
38 node1:26357,crdb-node2:26357,crdb-node3:26357 - --locality=region=
39 uteq,zone=nodo3 - --store=/cockroach/cockroach-data ports: - "
40 26259:26257" - "8082:8080" volumes: - crdb-node3-data:/cockroach/
41 cockroach-data networks: - crdb-network restart: unless-stopped
42
43 crdb-init: image: cockroachdb/cockroach:v25.2.3 container_name: crdb-
44 init depends_on: - crdb-node1 - crdb-node2 - crdb-node3 command: -
45 init - --insecure - --host=crdb-node1:26357 networks: - crdb-network
46 restart: "no"
47
48 networks: crdb-network: driver: bridge
49
50 volumes:
51   crdb-node1-data:
52   crdb-node2-data:

```



```
crdb-node3-data:
```

Listing 2: Configuración Docker Compose del clúster CockroachDB de tres nodos.

El despliegue se ejecutó mediante `docker compose up -d`; luego se consultó `docker compose ps`. Como muestra la Figura 2, los contenedores `crdb-node1`, `crdb-node2` y `crdb-node3` quedaron iniciados y con sus puertos publicados.

```
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipos_b> docker compose up -d
[*] up 8/8
✓ Network pe-u3-crdb-equipos_b_crdb-network Created
✓ Volume pe-u3-crdb-equipos_b_crdb-node1-data Created
✓ Volume pe-u3-crdb-equipos_b_crdb-node2-data Created
✓ Volume pe-u3-crdb-equipos_b_crdb-node3-data Created
✓ Container crdb-node1 Started
✓ Container crdb-node2 Started
✓ Container crdb-node3 Started
✓ Container crdb-init Started

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipos_b> docker compose ps
NAME                IMAGE                SERVICE    CREATED        STATUS        PORTS
crdb-node1          cockroachdb/cockroach:v25.2.3  "/cockroach/cockroach..."  crdb-node1    9 seconds ago  Up 7 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:26257->26257/tcp, [::]:26257->26257/tcp
crdb-node2          cockroachdb/cockroach:v25.2.3  "/cockroach/cockroach..."  crdb-node2    9 seconds ago  Up 7 seconds  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp, 0.0.0.0:26258->26257/tcp, [::]:26258->26257/tcp
crdb-node3          cockroachdb/cockroach:v25.2.3  "/cockroach/cockroach..."  crdb-node3    9 seconds ago  Up 7 seconds  0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp, 0.0.0.0:26259->26257/tcp, [::]:26259->26257/tcp
```

Figura 2: Inicialización de los servicios y verificación de los tres contenedores.

La comprobación visual se realizó en <http://localhost:8080/#/overview/list>. La Figura 3 registra tres nodos vivos, ninguno sospechoso, en drenaje o fuera de servicio. Asimismo, el panel muestra 75 rangos totales y cero rangos subreplicados o no disponibles en el momento de la captura.

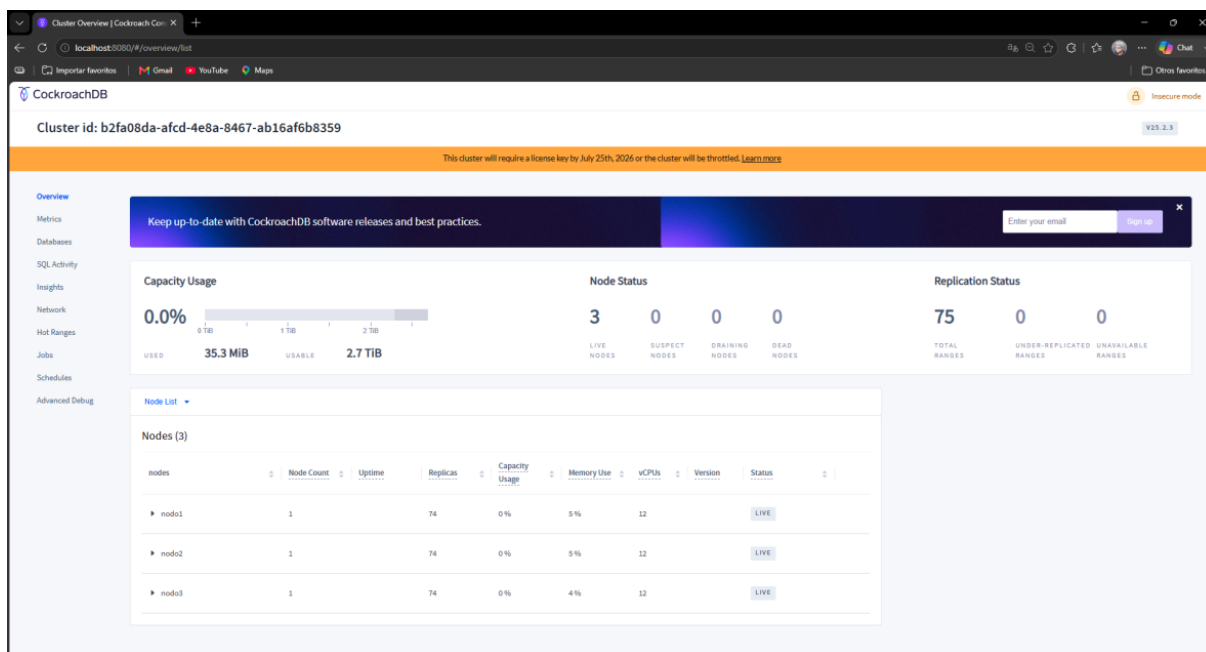


Figura 3: Dashboard de CockroachDB con los tres nodos en estado LIVE.

4.2. Creación del esquema

El contenido de `sql/01_schema.sql` se envió al nodo 1 mediante una canalización de PowerShell hacia el cliente SQL del contenedor. Después se ejecutó `SHOW TABLES` sobre

gestion_laboratorios. La salida de la Figura 4 confirma la creación de las diez tablas y de los índices definidos por el esquema.

```

Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> Get-Content -Raw .\sql\01_schema.sql |
>> docker compose exec -T crdb-node1 cockroach sql --insecure --host=crdb-node1:26257
CREATE DATABASE
SET
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> docker compose exec crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios `
>> -e "SHOW TABLES;"
 schema_name |      table_name      | type | owner | estimated_row_count | locality
-----|-----|-----|-----|-----|-----
 public      | asignacion_laboratorio | table | root  | 0 | NULL
 public      | detalle_asistencia    | table | root  | 0 | NULL
 public      | docente               | table | root  | 0 | NULL
 public      | equipo                | table | root  | 0 | NULL
 public      | estudiante            | table | root  | 0 | NULL
 public      | laboratorio           | table | root  | 0 | NULL
 public      | materia               | table | root  | 0 | NULL
 public      | persona               | table | root  | 0 | NULL
 public      | registro_asistencia   | table | root  | 0 | NULL
 public      | reporte_fallo         | table | root  | 0 | NULL
(10 rows)

Time: 28ms

```

Figura 4: Creación y verificación del esquema relacional de FUVV.

4.3. Aplicación y verificación del particionamiento

Tras crear el esquema se ejecutó `sql/02_partitions.sql`. La consulta `SHOW PARTITIONS FROM TABLE reporte_fallo` permitió comprobar las particiones y su configuración. La Figura 5 muestra los valores de partición y las políticas de zona asociadas.


```

Terminal Local +
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdB-equipo_b> Get-Content -Raw .\sql\01_queries.sql |
>> docker compose exec -T crdb-node1 cockroach sql --insecure --host=crdb-node1:26257
SET
id_reporte id_equipo id_docente descripcion estado fecha_reporte
id_reporte id_equipo descripcion estado fecha_reporte
id_reporte estado descripcion fecha_reporte codigo_equipo tipo_equipo marca modelo codigo_laboratorio nombre_laboratorio
id_laboratorio codigo_laboratorio nombre_laboratorio estado total_reportes
id_equipo codigo_equipo tipo_equipo marca modelo codigo_laboratorio nombre_laboratorio total_fallos fallos_pendientes fallos_en_proceso fallos_resueltos ultimo_reporte

```

Figura 7: Ejecución de las consultas de validación del dominio FUVV.

4.5. Carga masiva y validación de datos

El script `scripts/seed_data.py` generó información sintética para las diez tablas. Entre los valores configurados se encuentran 20 laboratorios, 50 docentes, 1 500 estudiantes, 400 equipos, 600 asignaciones, 1 200 registros de asistencia, 9 000 detalles y 2 500 reportes de fallo. El total informado por el proceso fue de 15 330 registros de prueba. La Figura 8 conserva la evidencia de la ejecución.

```

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdB-equipo_b> py .\scripts\seed_data.py
=====
CARGA MASIVA DE DATOS
=====
[OK] Datos anteriores eliminados.
[OK] Laboratorios insertados: 20
[OK] Personas insertadas: 1,550
[OK] Docentes insertados: 50
[OK] Estudiantes insertados: 1,500
[OK] Materias insertadas: 60
[OK] Equipos insertados: 400
[OK] Asignaciones insertadas: 600
[OK] Registros de asistencia insertados: 1,200
[OK] Detalles de asistencia insertados: 9,000
[OK] Reportes de fallos insertados: 2,500
=====
CARGA COMPLETADA: 15,330 REGISTROS
=====

```

Figura 8: Carga masiva de datos sintéticos mediante `seed_data.py`.

Posteriormente se agrupó `reporte_fallo` por estado para comprobar que las tres categorías contuvieran datos. La Figura 9 muestra resultados para `PENDIENTE`, `EN_PROCESO` y `RESUELTO`; por tanto, las particiones utilizadas por las consultas de validación no quedaron vacías.

```

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdB-equipo_b> docker compose exec crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios `
>> -e "SELECT estado, COUNT(*) FROM reporte_fallo GROUP BY estado;"
-----+-----
estado | count
-----+-----
EN_PROCESO | 834
PENDIENTE | 848
RESUELTO | 818
(3 rows)
Time: 8ms

```

Figura 9: Conteo de reportes insertados para cada estado.

4.6. Ejecución del benchmark del clúster

El programa `scripts/run_benchmark.py` abrió conexiones independientes a los puertos de los tres nodos. Se evaluaron cinco consultas: dos filtros por estado, una unión entre reportes, equipos y laboratorios, una agregación por laboratorio y una consulta de equipos con mayor cantidad de fallos. Antes de medir cada consulta se realizó una ejecución de calentamiento; a continuación se registraron diez repeticiones. El diseño produjo $3 \times 5 \times 10 = 150$ observaciones con tiempo de ejecución y número de filas.

```
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crd-b-equipos_b> py .\scripts\run_benchmark.py
=====
BENCHMARK DEL CLÚSTER COCKROACHDB
=====
Nodos: 3
Consultas: 5
Repeticiones por consulta: 10
Total de ejecuciones medidas: 150
[OK] nodo1 disponible en puerto 26257.
[OK] nodo2 disponible en puerto 26258.
[OK] nodo3 disponible en puerto 26259.

=====
PROBANDO NODO1 - PUERTO 26257
=====

Consulta: Q1_reportes_pendientes
Repetición 01: 4.5137 ms | 100 filas
Repetición 02: 3.5951 ms | 100 filas
Repetición 03: 3.7316 ms | 100 filas
Repetición 04: 3.4631 ms | 100 filas
Repetición 05: 5.8525 ms | 100 filas
Repetición 06: 4.8790 ms | 100 filas
Repetición 07: 4.7375 ms | 100 filas
Repetición 08: 5.0288 ms | 100 filas
Repetición 09: 4.3403 ms | 100 filas
Repetición 10: 5.3342 ms | 100 filas
Promedio: 4.5476 ms
Mínimo: 3.4631 ms
Máximo: 5.8525 ms

[OK] Resultados guardados en: C:\Users\franc\OneDrive\Escritorio\pe-u3-crd-b-equipos_b\evidencia\resultados.csv

=====
RESUMEN DEL BENCHMARK
=====
nodo1 | Q1_reportes_pendientes | promedio= 4.5476 ms | mínimo= 3.4631 ms | máximo= 5.8525 ms
nodo1 | Q2_reportes_en_proceso | promedio= 8.8541 ms | mínimo= 7.5107 ms | máximo= 14.2330 ms
nodo1 | Q3_reportes_resueltos_join | promedio= 17.4409 ms | mínimo= 14.3807 ms | máximo= 22.4880 ms
nodo1 | Q4_reportes_por_laboratorio | promedio= 17.7719 ms | mínimo= 14.6973 ms | máximo= 20.1295 ms
nodo1 | Q5_equipos_con_mas_fallos | promedio= 27.0879 ms | mínimo= 24.4448 ms | máximo= 30.5584 ms
nodo2 | Q1_reportes_pendientes | promedio= 8.8504 ms | mínimo= 8.2832 ms | máximo= 10.6618 ms
nodo2 | Q2_reportes_en_proceso | promedio= 11.6246 ms | mínimo= 8.2598 ms | máximo= 21.4566 ms
nodo2 | Q3_reportes_resueltos_join | promedio= 23.6420 ms | mínimo= 14.5268 ms | máximo= 42.8496 ms
nodo2 | Q4_reportes_por_laboratorio | promedio= 14.5794 ms | mínimo= 11.5585 ms | máximo= 18.1225 ms
nodo2 | Q5_equipos_con_mas_fallos | promedio= 23.4076 ms | mínimo= 19.8150 ms | máximo= 29.7770 ms
nodo3 | Q1_reportes_pendientes | promedio= 6.3566 ms | mínimo= 5.9079 ms | máximo= 7.1514 ms
nodo3 | Q2_reportes_en_proceso | promedio= 8.8426 ms | mínimo= 7.6607 ms | máximo= 12.2878 ms
nodo3 | Q3_reportes_resueltos_join | promedio= 13.9492 ms | mínimo= 11.6414 ms | máximo= 18.7394 ms
nodo3 | Q4_reportes_por_laboratorio | promedio= 16.1698 ms | mínimo= 14.1842 ms | máximo= 18.8488 ms
nodo3 | Q5_equipos_con_mas_fallos | promedio= 34.9991 ms | mínimo= 25.7319 ms | máximo= 41.6682 ms

=====
BENCHMARK COMPLETADO
=====
```

Figura 10: Ejecución de las cinco consultas sobre los tres nodos del clúster.

Los resultados brutos se almacenaron en `evidencia/resultados.csv`, como se observa en la Figura 11. Conservar cada repetición, en lugar de solo el promedio, permite revisar dispersión, valores extremos y estabilidad. Esta precaución es relevante porque una

comparación de SGBD debe hacer explícitos el protocolo experimental y la variabilidad de las mediciones [17], [18].

Linea	Medida	Valor
130	nodo3,26259,Q3_reportes_resueltos_join,7	14.4707,250
131	nodo3,26259,Q3_reportes_resueltos_join,10	11.6414,250
132	nodo3,26259,Q4_reportes_por_laboratorio,1	15.7447,60
133	nodo3,26259,Q4_reportes_por_laboratorio,2	14.1842,60
134	nodo3,26259,Q4_reportes_por_laboratorio,3	15.8684,60
135	nodo3,26259,Q4_reportes_por_laboratorio,4	16.8554,60
136	nodo3,26259,Q4_reportes_por_laboratorio,5	18.8488,60
137	nodo3,26259,Q4_reportes_por_laboratorio,6	16.9881,60
138	nodo3,26259,Q4_reportes_por_laboratorio,7	16.2319,60
139	nodo3,26259,Q4_reportes_por_laboratorio,8	15.7774,60
140	nodo3,26259,Q4_reportes_por_laboratorio,9	15.0738,60
141	nodo3,26259,Q4_reportes_por_laboratorio,10	16.1251,60
142	nodo3,26259,Q5_equipos_con_mas_fallos,1	33.2386,100
143	nodo3,26259,Q5_equipos_con_mas_fallos,2	39.5076,100
144	nodo3,26259,Q5_equipos_con_mas_fallos,3	41.008,100
145	nodo3,26259,Q5_equipos_con_mas_fallos,4	37.9252,100
146	nodo3,26259,Q5_equipos_con_mas_fallos,5	41.22,100
147	nodo3,26259,Q5_equipos_con_mas_fallos,6	35.6981,100
148	nodo3,26259,Q5_equipos_con_mas_fallos,7	41.6682,100
149	nodo3,26259,Q5_equipos_con_mas_fallos,8	25.7427,100
150	nodo3,26259,Q5_equipos_con_mas_fallos,9	28.251,100
151	nodo3,26259,Q5_equipos_con_mas_fallos,10	25.7319,100

Figura 11: Mediciones individuales conservadas en `resultados.csv`.

5. Resultados iniciales

La evidencia reunida confirma que el clúster se inicializó con tres nodos y que los tres aceptaron conexiones durante el procedimiento. El dashboard no registró rangos subreplicados ni indisponibles en el instante observado. El esquema de FUVV quedó creado con diez tablas y el conjunto de prueba alcanzó 15 330 registros. Asimismo, la partición de `reporte_fallo` por estado y las preferencias de leaseholder quedaron visibles mediante las consultas administrativas.

La Tabla 2 resume los promedios calculados a partir del archivo CSV actualmente almacenado en el repositorio. La captura de la Figura 10 documenta una ejecución del proceso, mientras que la tabla corresponde a la versión más reciente del CSV; por esta razón pueden existir diferencias menores entre los valores visibles en la terminal y los promedios aquí presentados.

Cuadro 2: Tiempo promedio por consulta y punto de acceso del clúster (ms).

Consulta	Nodo 1	Nodo 2	Nodo 3
Q1: reportes pendientes	3.7787	6.2745	5.8180
Q2: reportes en proceso	9.5671	7.7611	10.5002
Q3: reportes resueltos con uniones	9.1867	7.7361	5.8577
Q4: reportes agrupados por laboratorio	9.0778	8.2783	10.6400
Q5: equipos con más fallos	11.3778	10.8018	9.3147

Los resultados muestran que el punto de acceso con menor tiempo depende de la consulta. El nodo 1 obtuvo el menor promedio en Q1; el nodo 2 en Q2 y Q4; y el nodo 3 en Q3 y Q5. Esta variación es compatible con la existencia de leaseholders preferidos y con las diferencias entre filtros, uniones y agregaciones. Los planes de `EXPLAIN ANALYZE` presentados a continuación permiten identificar los operadores utilizados, aunque una atribución causal definitiva requiere además controlar el estado de la caché y repetir el experimento bajo condiciones equivalentes.

5.1. Análisis de los planes de ejecución

Las Figuras 12–16 documentan los planes reales de las cinco consultas ejecutadas desde el nodo 1. En todos los casos CockroachDB informó ejecución vectorizada y distribución local; sin embargo, los planes difieren en la cantidad de filas decodificadas y en los operadores requeridos. Q1 y Q2 emplearon una unión con el índice primario para recuperar las filas filtradas por estado. En Q1 el acceso de clave-valor se atendió en el nodo 1, mientras que Q2 muestra acceso al nodo 2, comportamiento compatible con las preferencias de leaseholder configuradas para esas particiones.


```
Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> Get-Content -Raw .\sql\04_q1_explain_analyze.sql |
>> docker compose exec -T crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios
info
planning time: 360µs
execution time: 4ms
distribution: local
vectorized: true
plan type: generic, reused
rows decoded from KV: 200 (18 KiB, 2 gRPC calls)
cumulative time spent in KV: 3ms
maximum memory usage: 196 KiB
DistSQL network usage: 0 B (0 messages)
regions: uteq
sql cpu time: 340µs
isolation level: serializable
priority: normal
quality of service: regular

- index join (streamer)
  sql nodes: n1
  kv nodes: n1
  regions: uteq
  actual row count: 100
  KV time: 2ms
  KV rows decoded: 100
  KV bytes read: 12 KiB
  KV gRPC calls: 1
  estimated max memory allocated: 160 KiB
  estimated max sql temp disk usage: 0 B
  sql cpu time: 254µs
  estimated row count: 100
  table: reporte_fallo@reporte_fallo_pkey

- scan
  sql nodes: n1"
  kv nodes: n1"
  regions: uteq"
  actual row count: 100"
  KV time: 1ms"
  KV rows decoded: 100"
  KV bytes read: 6.5 KiB"
  KV gRPC calls: 1"
  estimated max memory allocated: 20 KiB"
  sql cpu time: 86µs"
  estimated row count: 100 (4.0% of the table; stats collected 38 minutes ago)"
  table: reporte_fallo@idx_reporte_estado_fecha"
```

Figura 12: Plan de ejecución de Q1: reportes pendientes.


```
Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> Get-Content -Raw .\sql\05_q2_explain_analyze.sql |
>> docker compose exec -T crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios
info
planning time: 2ms
execution time: 13ms
distribution: local
vectorized: true
plan type: custom
rows decoded from KV: 400 (37 KiB, 2 gRPC calls)
cumulative time spent in KV: 12ms
maximum memory usage: 373 KiB
DistSQL network usage: 0 B (0 messages)
regions: uteq
sql cpu time: 843µs
isolation level: serializable
priority: normal
quality of service: regular

• index join (streamer)
  sql nodes: n1
  kv nodes: n2
  regions: uteq
  actual row count: 200
  KV time: 8ms
  KV rows decoded: 200
  KV bytes read: 24 KiB
  KV gRPC calls: 1
  estimated max memory allocated: 310 KiB
  estimated max sql temp disk usage: 0 B
  sql cpu time: 631µs
  estimated row count: 200
  table: reporte_fallo@reporte_fallo_pkey

  • scan
    sql nodes: n1"
    kv nodes: n2"
    regions: uteq"
    actual row count: 200"
    KV time: 4ms"
    KV rows decoded: 200"
    KV bytes read: 13 KiB"
    KV gRPC calls: 1"
    estimated max memory allocated: 40 KiB"
    sql cpu time: 212µs"
    estimated row count: 200 (8.0% of the table; stats collected 39 minutes ago)"
    table: reporte_fallo@idx_reporte_estado_fecha"
```

Figura 13: Plan de ejecución de Q2: reportes en proceso.

Q3 incorporó una unión hash y un operador *top-k* para combinar y ordenar los reportes resueltos con la información relacionada; su plan decodificó 1 638 filas. Q4 procesó 2 920 filas mediante una unión por búsqueda, una agrupación hash y un ordenamiento para calcular los reportes agrupados por laboratorio. Estas operaciones explican que ambas consultas realicen más trabajo que los filtros directos Q1 y Q2.

```
Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> Get-Content -Raw .\sql\06_q3_explain_analyze.sql |
>> docker compose exec -T crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios
info
planning time: 2ms
execution time: 16ms
distribution: local
vectorized: true
plan type: custom
rows decoded from KV: 1,638 (178 KiB, 5 gRPC calls)
cumulative time spent in KV: 13ms
maximum memory usage: 1.1 MiB
DistSQL network usage: 0 B (0 messages)
regions: uteq
sql cpu time: 3ms
isolation level: serializable
priority: normal
quality of service: regular

• sort
  sql nodes: n1
  regions: uteq
  actual row count: 250
  execution time: 71µs
  estimated max memory allocated: 180 KiB
  sql cpu time: 71µs
  estimated row count: 13
  order: -fecha_reporte

• hash join
  sql nodes: n1"
  regions: uteq"
  actual row count: 250"
  execution time: 351µs"
  estimated max memory allocated: 300 KiB"
  sql cpu time: 350µs"
  estimated row count: 13"
  equality: (id_equipo) = (id_equipo)"
  right cols are key"
  "
  "
  • top-k"
  sql nodes: n1"
  regions: uteq"
  actual row count: 250"
  execution time: 344µs"
  estimated max memory allocated: 100 KiB"
  sql cpu time: 344µs"
```

Figura 14: Plan de ejecución de Q3: reportes resueltos con uniones.

```

Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crd-db-equipos_b> Get-Content -Raw .\sql\07_q4_explain_analyze.sql |
>> docker compose exec -T crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios
info
planning time: 2ms
execution time: 30ms
distribution: local
vectorized: true
plan type: custom
rows decoded from KV: 2,920 (235 KiB, 4 gRPC calls)
cumulative time spent in KV: 24ms
maximum memory usage: 1.3 MiB
DistSQL network usage: 0 B (0 messages)
regions: uteq
sql cpu time: 5ms
isolation level: serializable
priority: normal
quality of service: regular

- sort
  sql nodes: n1
  regions: uteq
  actual row count: 60
  execution time: 54µs
  estimated max memory allocated: 20 KiB
  sql cpu time: 54µs
  estimated row count: 3
  order: -count_rows,+codigo,+estado
- group (hash)
  sql nodes: n1"
  regions: uteq"
  actual row count: 60"
  execution time: 2ms"
  estimated max memory allocated: 1.1 MiB"
  estimated max sql temp disk usage: 0 B"
  sql cpu time: 2ms"
  estimated row count: 3"
  group by: estado, codigo"
- lookup join (streamer)"
  sql nodes: n1"
  kv nodes: n1"
  regions: uteq"
  actual row count: 2,500"
  KV time: 19ms"
  KV rows decoded: 2,500"

```

Figura 15: Plan de ejecución de Q4: reportes agrupados por laboratorio.

Q5 fue el plan con mayor volumen decodificado, con 3 320 filas. La consulta aplicó agrupación hash, filtrado y selección *top-k* para identificar los equipos con más fallos. La combinación de agregación y ordenamiento resulta coherente con que Q5 presentara los promedios más altos del conjunto en los tres puntos de acceso.

```
Terminal Local x + v
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdp-equipo_b> Get-Content -Raw .\sql\08_q5_explain_analyze.sql
>> docker compose exec -T crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios
info
planning time: 2ms
execution time: 28ms
distribution: local
vectorized: true
plan type: custom
rows decoded from KV: 3,320 (276 KiB, 5 gRPC calls)
cumulative time spent in KV: 21ms
maximum memory usage: 1.3 MiB
DistSQL network usage: 0 B (0 messages)
regions: uteq
sql cpu time: 7ms
isolation level: serializable
priority: normal
quality of service: regular

. top-k
  sql nodes: n1
  regions: uteq
  actual row count: 100
  execution time: 194µs
  estimated max memory allocated: 70 KiB
  sql cpu time: 193µs
  estimated row count: 7
  order: -count_rows,-max
  k: 100

. filter
  sql nodes: n1"
  regions: uteq"
  actual row count: 400"
  execution time: 8µs"
  sql cpu time: 8µs"
  estimated row count: 7"
  filter: count_rows >= 1"
  "
  . group (hash)"
  sql nodes: n1"
  regions: uteq"
  actual row count: 400"
  execution time: 3ms"
  estimated max memory allocated: 340 KiB"
  sql cpu time: 3ms"
  estimated row count: 20"
```

Figura 16: Plan de ejecución de Q5: equipos con mayor cantidad de fallos.

5.2. Prueba de tolerancia a fallos y reintegración

La Figura 17 representa la secuencia completa del experimento. El flujo comienza con la verificación de los tres nodos y la comunicación normal del líder con sus seguidores; después muestra la detención del nodo 2, la continuidad de las lecturas, la confirmación de una escritura mediante el quórum formado por los nodos 1 y 3 y, finalmente, la reconexión del nodo detenido, la actualización de las entradas faltantes del log y la recuperación del estado activo de los tres nodos.

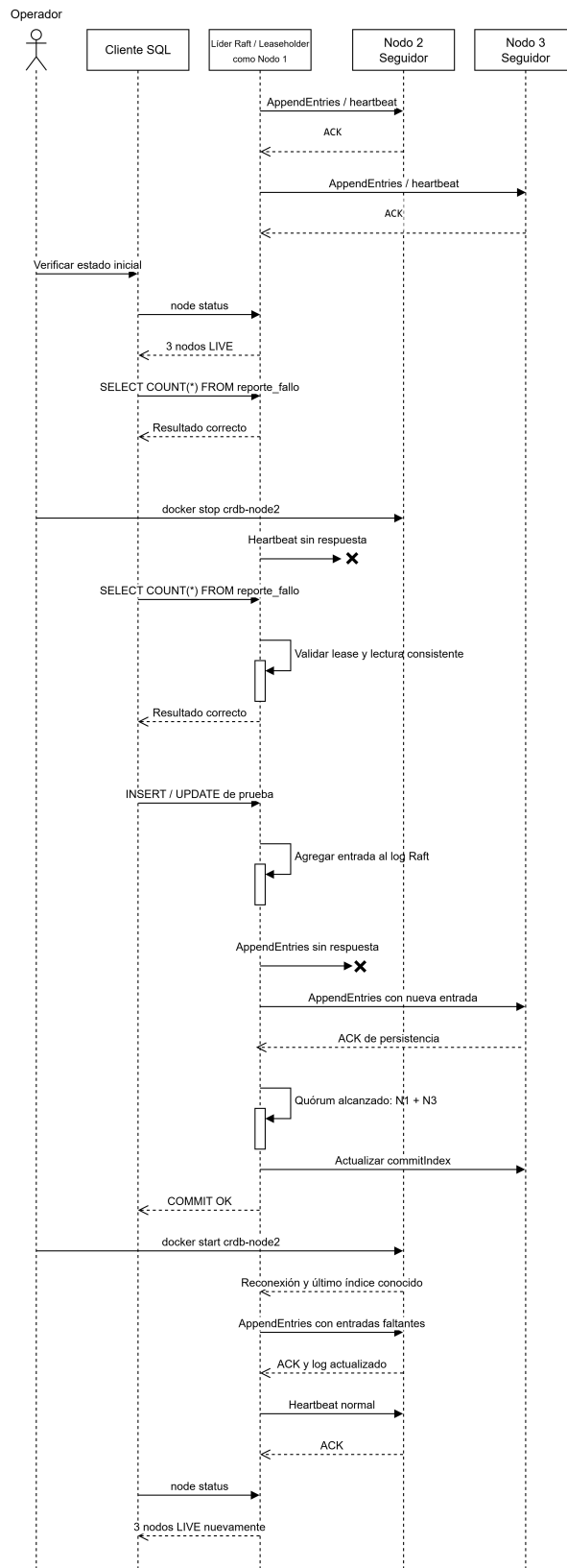


Figura 17: Diagrama de secuencia de la prueba de tolerancia a fallos y reintegración del nodo 2.

Antes de provocar la falla se verificó el estado del clúster mediante `cockroach`

node status. La Figura 18 muestra que los tres nodos estaban disponibles y activos, con las localidades lógicas nodo1, nodo2 y nodo3; esta comprobación estableció la condición inicial necesaria para interpretar la prueba.

```
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdp-equipa_b> docker compose exec crdb-node1 cockroach node status `
>> --insecure `
>> --host=crdb-node1:26257
```

id	address	sql_address	build	started_at	updated_at	locality	is_available	is_live
1	crdb-node1:26257	crdb-node1:26257	v25.2.3	2026-07-19 01:59:29.175159 +0000 UTC	2026-07-19 01:59:35.267027 +0000 UTC	region=uteq,zone=nodo1	true	true
2	crdb-node2:26257	crdb-node2:26257	v25.2.3	2026-07-19 01:59:29.148028 +0000 UTC	2026-07-19 01:59:35.251755 +0000 UTC	region=uteq,zone=nodo2	true	true
3	crdb-node3:26257	crdb-node3:26257	v25.2.3	2026-07-19 01:59:29.148043 +0000 UTC	2026-07-19 01:59:35.243154 +0000 UTC	region=uteq,zone=nodo3	true	true

(3 rows)

Figura 18: Verificación del estado inicial de los tres nodos del clúster.

La distribución previa a la falla se examinó con `SHOW RANGES FROM TABLE reporte_fallo`. En la Figura 19 se observan 14 rangos con réplicas asignadas a los tres nodos, lo cual confirma que la tabla contaba con copias distribuidas antes de detener uno de los integrantes del clúster.

```
PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdp-equipa_b> docker compose exec crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios `
>> -e "SHOW RANGES FROM TABLE reporte_fallo;"
```

start_key	end_key	range_id	replicas	replica_localities	voting_replicas	non_voting_replicas	learner_replicas	spl
<before:/Table/100>	_/8	81	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/8	_/14/EN_PROCESO*	94	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	2026-0
7-10 21:41:00.425210	_/14/EN_PROCESO*	104	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/14/EN_PROCESO*/PrefixEnd	_/14/PENDIENTE*	105	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/14/PENDIENTE*	_/14/PENDIENTE*/PrefixEnd	106	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/14/PENDIENTE*/PrefixEnd	_/14/RESUELTO*	107	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/14/RESUELTO*	_/14/RESUELTO*/PrefixEnd	108	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/14/RESUELTO*/PrefixEnd	_/16/EN_PROCESO*	109	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/EN_PROCESO*	_/16/EN_PROCESO*/PrefixEnd	110	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/EN_PROCESO*/PrefixEnd	_/16/PENDIENTE*	111	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/PENDIENTE*	_/16/PENDIENTE*/PrefixEnd	112	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/PENDIENTE*/PrefixEnd	_/16/RESUELTO*	113	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/RESUELTO*	_/16/RESUELTO*/PrefixEnd	114	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL
_/16/RESUELTO*/PrefixEnd	<after:/Max>	115	{1,2,3}	{region=uteq,zone=nodo1},{region=uteq,zone=nodo2},{region=uteq,zone=nodo3}	{1,3,2}	{}	{}	NULL

(14 rows)

Time: 34ms

Figura 19: Distribución inicial de los rangos de la tabla reporte_fallo.

La prueba de disponibilidad consistió en consultar la cantidad de registros de `reporte_fallo`, detener el contenedor `crdb-node2` y repetir la misma operación desde un nodo superviviente. Como muestra la Figura 20, la consulta devolvió 2,500 registros tanto antes como después de la caída. El clúster mantuvo el servicio porque las dos réplicas restantes conservaron la mayoría requerida por el grupo de consenso.


```

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> docker compose exec crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios `
>> -e "SELECT COUNT(*) FROM reporte_fallo;"
count
-----
2500
(1 row)

Time: 6ms

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> docker stop crdb-node2
crdb-node2

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> docker compose exec crdb-node1 cockroach sql `
>> --insecure `
>> --host=crdb-node1:26257 `
>> --database=gestion_laboratorios `
>> -e "SELECT COUNT(*) FROM reporte_fallo;"
count
-----
2500
(1 row)

Time: 4ms

```

Figura 20: Consulta exitosa durante la caída controlada del nodo 2.

Finalmente, se inició nuevamente el contenedor `crdb-node2` y se midió la ejecución de la verificación de estado. La Figura 21 registra el reinicio y un tiempo aproximado de 1.09 segundos para completar el comando de comprobación. Esta medición representa el tiempo observado por el procedimiento de verificación, por lo que no debe interpretarse por sí sola como una medición interna exacta de la recuperación de todas las réplicas.

```

Terminal Local + v
>> --database=gestion_laboratorios
>> -e "SHOW RANGES FROM TABLE reporte_fallo;"
start_key | end_key | range_id | replicas | replica_localities | voting_replicas | non_voting_replicas | learner_replicas | sp
-----
<before://table/1d0> | ..8 | 81 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..8 | ..14/"EN_PROCESO" | 94 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
7-10 21:41:08.425210 | ..14/"EN_PROCESO" | ..14/"EN_PROCESO"/PrefixEnd | 104 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..14/"EN_PROCESO"/PrefixEnd | ..14/"PENDIENTE" | 105 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..14/"PENDIENTE" | ..14/"PENDIENTE"/PrefixEnd | 106 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..14/"PENDIENTE"/PrefixEnd | ..14/"RESUELTO" | 107 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..14/"RESUELTO" | ..14/"RESUELTO"/PrefixEnd | 108 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..14/"RESUELTO"/PrefixEnd | ..16/"EN_PROCESO" | 109 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"EN_PROCESO" | ..16/"EN_PROCESO"/PrefixEnd | 110 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"EN_PROCESO"/PrefixEnd | ..16/"PENDIENTE" | 111 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"PENDIENTE" | ..16/"PENDIENTE"/PrefixEnd | 112 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"PENDIENTE"/PrefixEnd | ..16/"RESUELTO" | 113 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"RESUELTO" | ..16/"RESUELTO"/PrefixEnd | 114 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
..16/"RESUELTO"/PrefixEnd | <after://Max> | 115 | {1,2,3} | {"region=uteq,zone=nodo1","region=uteq,zone=nodo2","region=uteq,zone=nodo3"} | {1,3,2} | {} | {} | {} | NULL
(14 rows)

Time: 19ms

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> docker start crdb-node2
crdb-node2

PS C:\Users\franc\OneDrive\Escritorio\pe-u3-crdb-equipo_b> Measure-Command {
>> docker compose exec crdb-node1 cockroach node status --insecure --host=crdb-node1:26257
>> }

Days : 0
Hours : 0
Minutes : 0
Seconds : 1
Milliseconds : 85
Ticks : 10653020
TotalDays : 1.205379E-05
TotalHours : 0.000301473
TotalMinutes : 0.01808838
TotalSeconds : 1.0853028
TotalMilliseconds : 1085.3028

```

Figura 21: Reinicio y tiempo observado de reintegración del nodo 2.

Alcance de la prueba de fallos. La prueba controlada demuestra continuidad de lectura frente a la caída de un nodo y documenta su reinicio. La captura final incluye la

consulta de rangos realizada con el nodo 2 detenido y el inicio posterior del contenedor. El tiempo registrado corresponde a la ejecución del procedimiento de comprobación y no a una medición interna exacta de la recuperación de cada réplica; por ello, el video continuo y futuras mediciones repetidas permiten complementar la interpretación del experimento.

6. Comparación de rendimiento con nodo único

Para completar la comparación se levantó una instancia independiente mediante `docker-compose.single.yml` y se cargó el mismo esquema con 15 330 registros generados mediante la semilla determinista 2026. Las cinco consultas y el número de repeticiones se mantuvieron sin cambios. El tiempo del clúster de la Tabla 3 corresponde al promedio de las treinta observaciones disponibles por consulta, diez desde cada punto de acceso, mientras que el tiempo de nodo único corresponde a diez observaciones. Esta diferencia de tamaño muestral se declara para que la comparación sea reproducible y no se interprete como una prueba estadística concluyente [8], [17].

El factor de mejora se definió como la relación entre el tiempo promedio del nodo único y el tiempo promedio del clúster:

$$F_i = \frac{T_{\text{único},i}}{T_{\text{clúster},i}}. \quad (7)$$

Un valor $F_i > 1$ indica menor latencia en el clúster; un valor $F_i < 1$ indica que la instancia única fue más rápida. Esta convención expresa una relación de latencias y no implica que el sistema pueda procesar exactamente F_i veces más solicitudes concurrentes, pues el rendimiento total también depende de carga, paralelismo, caché, red y contención [17], [18].

Cuadro 3: Comparación de las cinco consultas en el clúster y en nodo único.

Consulta	Clúster (ms)	Único (ms)	F_i	Interpretación
Q1: reportes pendientes	5.2904	5.4985	1.039	Ventaja leve del clúster; filtro directo asociado con la partición pendiente.
Q2: reportes en proceso	9.2761	6.3010	0.679	Nodo único más rápido; la coordinación y el acceso remoto reducen la ventaja distribuida.
Q3: reportes resueltos con uniones	7.5935	12.7959	1.685	Mayor ventaja del clúster; el plan combina uniones y ordenamiento sobre más filas.
Q4: reportes agrupados por laboratorio	9.3320	7.9940	0.857	Nodo único más rápido; la agregación no compensó el costo del clúster.
Q5: equipos con más fallos	10.4981	15.2845	1.456	Ventaja del clúster en la consulta con mayor volumen decodificado.

El promedio no ponderado de las cinco consultas fue 8.3980 ms para el clúster y 9.5750 ms para el nodo único, una reducción global aproximada de 12.3 % respecto del

tiempo del nodo único. No obstante, el resultado agregado oculta diferencias importantes: Q1 mostró una ventaja de apenas 3.9% para el clúster, mientras que Q3 y Q5 alcanzaron factores de 1.685 y 1.456. En sentido contrario, Q2 y Q4 favorecieron al nodo único. Los planes de ejecución ayudan a interpretar esta variación: Q1 y Q2 utilizaron uniones con el índice primario, Q3 incorporó unión hash y *top-k*, Q4 combinó unión por búsqueda, agrupación y ordenamiento, y Q5 procesó el mayor número de filas mediante agrupación, filtrado y selección *top-k*. La distribución puede beneficiar operaciones con mayor trabajo y acceso a rangos distintos, pero también introduce coordinación que resulta visible cuando la consulta es pequeña o se resuelve eficientemente dentro de una sola instancia [3], [18].

Los datos permiten concluir que el clúster no es universalmente más rápido. Su principal justificación para FUVV se encuentra en la continuidad operativa, la replicación y la capacidad de escalar, no únicamente en reducir la latencia de una consulta aislada. Para fortalecer el análisis sería necesario repetir ambos escenarios en la misma máquina bajo ventanas temporales controladas, registrar dispersión y percentiles, alternar el orden de ejecución y someter el sistema a concurrencia creciente. Estas precauciones son necesarias porque una comparación de SGBD depende del protocolo, la configuración, la caché y la carga, y no puede generalizarse solo a partir de promedios de una ejecución [8], [17].

7. Preguntas de análisis

7.1. ¿Cómo garantiza Raft la consistencia cuando falla un nodo?

CockroachDB divide el espacio de claves en rangos y mantiene varias réplicas de cada rango. Cada grupo de réplicas participa en un protocolo de consenso inspirado en Raft, en el que una réplica actúa como líder del grupo y ordena las propuestas dentro de un log replicado. El leaseholder atiende y coordina muchas solicitudes SQL, aunque su función no debe confundirse por completo con el liderazgo de Raft. Cuando llega una escritura, la propuesta recibe una posición en el log y se envía a las demás réplicas. Con tres réplicas, una mayoría está formada por dos; la entrada solo se considera comprometida cuando obtiene las confirmaciones necesarias. Esta regla impide que una réplica aislada confirme unilateralmente una historia incompatible con la mayoría y permite conservar un orden común de operaciones comprometidas [3], [11].

Si falla un seguidor, el líder todavía puede acordar nuevas entradas con el seguidor restante. Si falla el líder, las réplicas disponibles realizan una elección y solo un candidato que pueda reunir la mayoría asume el nuevo término. Las restricciones de voto, término y coincidencia de prefijos evitan que dos historiales incompatibles se comprometan simultáneamente. Una operación que no alcanzó la mayoría antes del fallo puede reintentarse o abortarse, pero no se presenta como confirmada. En cambio, una entrada ya comprometida permanece en la historia válida porque cualquier mayoría futura comparte al menos una réplica con la mayoría que la confirmó. Cuando el nodo detenido regresa, compara su último índice y recibe las entradas faltantes antes de reincorporarse normalmente al grupo

[3], [9].

En la práctica de FUVV, la tabla `reporte_fallo` conservó tres réplicas distribuidas entre `nodo1`, `nodo2` y `nodo3`. Al detener `crdb-node2`, los nodos 1 y 3 conservaron el quórum de dos y la consulta `SELECT COUNT(*)` continuó devolviendo 2 500 registros. La consistencia no se obtuvo porque todos los nodos estuvieran disponibles, sino porque las operaciones se limitaron a la historia aceptada por una mayoría. Si hubieran fallado dos nodos o se hubiera perdido la comunicación entre las dos réplicas restantes, el sistema habría rechazado las operaciones que no pudieran acordarse. Esa decisión sacrifica disponibilidad sin mayoría para evitar resultados divergentes, comportamiento coherente con la prioridad de consistencia seleccionada para reservas y reportes del PFC [3], [12].

7.2. CockroachDB frente a Cassandra desde la consistencia y PACELC

CockroachDB ofrece transacciones con aislamiento serializable y busca que el resultado concurrente sea equivalente a algún orden secuencial válido. Sus rangos se replican mediante consenso y las escrituras requieren el acuerdo de una mayoría, por lo que una operación confirmada queda asociada con una historia consistente. Cassandra adopta un modelo de columnas anchas orientado a distribución y disponibilidad, con niveles de consistencia configurables para lecturas y escrituras. Según la combinación elegida, una aplicación puede favorecer respuestas rápidas y tolerar que ciertas réplicas devuelvan temporalmente valores antiguos; también puede solicitar niveles más fuertes, aunque el comportamiento y el costo dependen del factor de replicación y de los quórums configurados. Por ello, no es correcto reducir Cassandra a una única garantía eventual ni afirmar que siempre prioriza disponibilidad: su consistencia es ajustable y debe analizarse junto con la configuración concreta [13], [14].

PACELC amplía la discusión de CAP. Durante una partición de red se debe decidir entre disponibilidad y consistencia; cuando no existe partición, todavía aparece una decisión entre latencia y consistencia. CockroachDB se aproxima a una posición PC/EC: ante la pérdida de mayoría rechaza las operaciones que no puede ordenar de forma segura y, durante el funcionamiento normal, acepta el costo de coordinación necesario para mantener garantías fuertes. Una configuración de Cassandra orientada a respuestas locales y consistencia eventual se acercaría a PA/EL, aunque otros niveles de lectura y escritura pueden desplazar ese equilibrio. El modelo permite entender que la diferencia no aparece únicamente durante fallos; también se manifiesta como latencia adicional en las operaciones normales que coordinan réplicas `noudoust2022quorum`, [12].

Para FUVV se elegiría CockroachDB. Una reserva duplicada, la asignación simultánea del mismo laboratorio o una actualización contradictoria del estado de un equipo constituyen errores funcionales que pueden afectar a docentes, estudiantes y administradores. La serializabilidad facilita razonar sobre restricciones, claves y transacciones entre varias tablas sin trasladar a la aplicación toda la reconciliación de versiones. Cassandra

sería atractiva si el objetivo dominante fuera recibir grandes volúmenes de telemetría donde una lectura ligeramente atrasada fuese aceptable y la disponibilidad de escritura local tuviera prioridad. Sin embargo, el núcleo de reservas y control operativo necesita una visión consistente. Los resultados experimentales muestran que esta elección no siempre minimiza la latencia, pero el costo adicional es razonable cuando protege reglas esenciales del dominio [3], [14].

7.3. Fragmentación horizontal explícita para la tabla principal del PFC

La relación principal seleccionada fue `reporte_fallo` porque representa eventos operativos con un ciclo de vida claro y constituye una fuente frecuente de consultas de monitoreo. La estrategia implementada utiliza `PARTITION BY LIST` sobre el atributo `estado` y define las particiones `reporte_pendiente`, `reporte_en_proceso` y `reporte_resuelto`. Para que la clave de partición forme parte del orden de almacenamiento, la clave primaria se definió como `(estado, id_reporte)`, mientras una restricción única conserva la identificación lógica del reporte. De manera formal, si R representa `reporte_fallo`, los fragmentos son $R_P = \sigma_{estado='PENDIENTE'}(R)$, $R_E = \sigma_{estado='EN_PROCESO'}(R)$ y $R_R = \sigma_{estado='RESUELTO'}(R)$ [1], [7].

La estrategia satisface completitud porque todo estado permitido por el esquema pertenece a una de las tres condiciones; satisface reconstrucción porque $R = R_P \cup R_E \cup R_R$; y satisface disjunción porque una fila solo puede tener un valor de estado a la vez. Cada partición mantiene tres réplicas, una restringida a cada zona lógica, y establece una preferencia de leaseholder: pendientes en `nodo1`, en proceso en `nodo2` y resueltos en `nodo3`. La partición no asigna una única copia exclusiva a cada nodo; distribuye la ubicación preferida de atención mientras la replicación conserva disponibilidad. Esta distinción evita confundir fragmentación lógica, ubicación del leaseholder y número físico de réplicas [1], [3].

La clave `estado` se justifica por las consultas operativas del PFC: el personal puede revisar incidencias pendientes, dar seguimiento a las que están en proceso y consultar el historial resuelto. La prueba cargó 2 500 reportes distribuidos entre las tres categorías y permitió observar rangos y preferencias asociados. La principal limitación es que `estado` es mutable; al resolver una incidencia, la fila cambia de espacio particionado, lo que puede producir movimiento de claves y trabajo adicional. Si el sistema creciera por sedes geográficas, una alternativa sería particionar por identificador de laboratorio o región y usar índices por estado, reduciendo movimientos durante el ciclo de vida. La elección definitiva debe basarse en frecuencia de filtros, distribución de carga, costo de actualización y afinidad con las uniones, no solo en una división conceptualmente sencilla [7], [8].

7.4. Atomicidad de transacciones multitable y relación entre commit y Raft

Una operación de FUVV puede afectar varias tablas y rangos, por ejemplo crear una reserva, registrar su asignación de laboratorio y actualizar la disponibilidad de recursos. La atomicidad exige que todos esos cambios se confirmen como una sola unidad o que ninguno quede visible. CockroachDB ejecuta estas operaciones dentro de una transacción distribuida y crea intenciones de escritura en los rangos participantes. Cada rango replica su parte mediante consenso: Raft ordena y hace durables las propuestas aceptadas por la mayoría de las réplicas de ese rango. Sin embargo, Raft por sí solo no resuelve la atomicidad entre rangos diferentes; garantiza acuerdo dentro de cada grupo. La decisión global requiere además un protocolo transaccional que coordine el estado común de commit o abort [3], [10].

Conceptualmente, el flujo se relaciona con el commit en dos fases. Durante la preparación, los participantes registran que pueden confirmar y conservan los cambios sin hacerlos visibles como resultados definitivos. El coordinador solo confirma cuando se cumplen las condiciones para todos; si un participante rechaza la operación, se aborta la transacción y se eliminan o resuelven las intenciones. CockroachDB incorpora optimizaciones como el commit paralelo y estados transaccionales replicados para reducir viajes de red y evitar que el camino normal dependa de una secuencia rígida de mensajes. Aun así, la propiedad esencial permanece: una respuesta exitosa solo se entrega cuando existe evidencia durable suficiente para resolver todas las escrituras de manera uniforme. El registro y las intenciones permiten que otros nodos determinen posteriormente el resultado incluso si el coordinador original deja de responder [3], [9].

Ante un fallo, cada modificación ya aceptada permanece protegida por el log replicado de su rango. Si el coordinador desaparece, otro nodo puede consultar el estado de la transacción y continuar su resolución; no necesita confiar en la memoria volátil de la máquina fallida. Si no existe quórum para alguno de los rangos participantes, la operación no puede confirmarse de forma segura y debe esperar, reintentarse o abortarse según su estado. Para FUVV, este comportamiento evita que una asignación exista sin su reserva asociada o que la disponibilidad de un laboratorio cambie sin registrar la operación correspondiente. El costo es una mayor coordinación que en una escritura independiente, pero la alternativa sería trasladar a la aplicación la detección y compensación de estados parciales, con mayor complejidad y riesgo de inconsistencias [2], [10].

8. Conclusiones individuales

8.1. Conclusión de José Alejandro Lozano Morales

El desarrollo de esta práctica me permitió comprender que una base de datos distribuida no se limita a ejecutar el mismo motor en varios contenedores. Su funcionamiento

depende de decisiones coordinadas sobre esquema, claves, fragmentación, replicación, consistencia y recuperación. El análisis bibliográfico fue especialmente importante para distinguir conceptos que suelen confundirse, como fragmentación lógica, ubicación preferida del leaseholder y copia física de una réplica. En FUVV, la partición de `reporte_fallo` por estado facilita consultas operativas, pero también revela que una clave mutable puede generar movimientos cuando el reporte avanza en su ciclo de vida. La revisión de fuentes y DOI me ayudó además a reconocer que una referencia solo es útil cuando puede verificarse y realmente respalda la afirmación asociada. Los resultados comparativos mostraron que distribuir no equivale automáticamente a acelerar: el clúster fue más rápido en tres consultas, mientras el nodo único obtuvo mejores tiempos en dos. Por tanto, considero que la principal ventaja para el PFC es conservar disponibilidad y consistencia frente a fallos, junto con una ruta de crecimiento horizontal. Como aprendizaje final, cualquier decisión de arquitectura debe justificarse simultáneamente con teoría verificable, evidencia reproducible y necesidades concretas del dominio.

9. Declaración de uso de inteligencia artificial generativa

El equipo utilizó ChatGPT como herramienta de apoyo para realizar comprobaciones y proponer arreglos durante el desarrollo del trabajo. Su asistencia se aplicó a la revisión de código SQL, Python, Docker Compose y LaTeX; la detección de errores de rutas, sintaxis, estructura y compilación; la organización y comprobación de referencias bibliográficas, DOI, metadatos y formato IEEE; la revisión de coherencia entre la rúbrica y las secciones del informe; la comprobación conceptual y visual del diagrama de secuencia; y la mejora de redacción, tablas, pies de figura y referencias cruzadas. ChatGPT también se empleó para contrastar que las interpretaciones fueran compatibles con los archivos CSV y con las evidencias capturadas. La herramienta no ejecutó por cuenta propia el experimento ni sustituyó las decisiones académicas del equipo: los integrantes seleccionaron el PFC, configuraron el entorno, ejecutaron las pruebas, verificaron las fuentes disponibles, revisaron las propuestas y asumieron la responsabilidad final por el código, los resultados, el análisis y el contenido entregado.

9.1. Conclusión de Andy Paul Sanchez Pilaloa

La construcción y documentación del clúster me permitió relacionar la configuración visible de Docker con mecanismos internos de CockroachDB. Los tres nodos comparten una red, pero mantienen almacenamiento persistente, localidades y puntos de acceso independientes; sobre esa infraestructura, los rangos y sus réplicas permiten que el sistema continúe respondiendo cuando una máquina deja de estar disponible. La prueba de caída del nodo 2 fue el resultado más claro: el conteo de `reporte_fallo` devolvió los mismos 2 500 registros antes y después de detener el contenedor, lo cual hizo observable el efecto práctico

del quórum. También aprendí que documentar una prueba exige algo más que incluir capturas. Es necesario explicar la condición inicial, el comando ejecutado, el resultado, la limitación de la medición y la relación con el modelo teórico. La elaboración del informe en LaTeX reforzó la importancia de usar referencias cruzadas, tablas formales, ecuaciones y código legible para que otra persona pueda reconstruir el procedimiento. Para FUVV, esta experiencia demuestra que la continuidad del servicio debe diseñarse desde la capa de datos y no añadirse al final. Una reserva o un reporte crítico necesitan seguir disponibles sin sacrificar la corrección de su estado.

Bibliografía

- [1] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Switzerland: Springer, 2020, ISBN: 978-3-030-26252-5. DOI: 10.1007/978-3-030-26253-2.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [3] R. Taft et al., «CockroachDB: The Resilient Geo-Distributed SQL Database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ACM, 2020, págs. 1493-1509. DOI: 10.1145/3318464.3386134.
- [4] N. VanBenschoten et al., «Enabling the Next Generation of Multi-Region Applications with CockroachDB,» en *Proceedings of the 2022 International Conference on Management of Data*, ACM, 2022, págs. 2312-2325. DOI: 10.1145/3514221.3526053.
- [5] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.^a ed. Maastricht, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6.
- [6] H. I. Abdalla, A. A. Amer y S. D. Ravana, «On Hierarchical Clustering-Based Approach for RDDBS Design,» *Journal of Big Data*, vol. 10, n.º 1, pág. 172, 2023. DOI: 10.1186/s40537-023-00849-7.
- [7] K. Danach, A. H. Khalaf, A. Rammal y H. Harb, «Enhancing DDBMS Performance through RFO-SVM Optimized Data Fragmentation: A Strategic Approach to Machine Learning Enhanced Systems,» *Applied Sciences*, vol. 14, n.º 14, pág. 6093, 2024. DOI: 10.3390/app14146093.
- [8] J. M. Schäfer, U. Sax y L. Wiese, «Benchmarking a Distributed Database Design That Supports Patient Cohort Identification,» en *Proceedings of the 24th Symposium on International Database Engineering and Applications*, ACM, 2020, págs. 1-8. DOI: 10.1145/3410566.3410608.
- [9] S. Gupta y M. Sadoghi, «Efficient and Non-Blocking Agreement Protocols,» *Distributed and Parallel Databases*, vol. 38, n.º 2, págs. 287-333, 2020. DOI: 10.1007/s10619-019-07267-w.

- [10] M. de Heus, K. Psarakis, M. Fragkoulis y A. Katsifodimos, «Transactions across Serverless Functions Leveraging Stateful Dataflows,» *Information Systems*, vol. 108, págs. 102015, 2022. DOI: 10.1016/j.is.2022.102015.
- [11] N. V. Lewchenko, G. Kaki y B.-Y. E. Chang, «Bolt-On Strong Consistency: Specification, Implementation, and Verification,» *Proceedings of the ACM on Programming Languages*, vol. 9, n.º OOPSLA1, págs. 1604-1631, 2025. DOI: 10.1145/3720502.
- [12] S. Gilbert y N. Lynch, «Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601.
- [13] P. Viotti y M. Vukolić, «Consistency in Non-Transactional Distributed Storage Systems,» *ACM Computing Surveys*, vol. 49, n.º 1, págs. 1-34, 2016. DOI: 10.1145/2926965.
- [14] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of Data Consistency Levels in Cloud-Based NoSQL for Data-Intensive Applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, págs. 158, 2024. DOI: 10.1186/s13677-024-00716-7.
- [15] G. DeCandia et al., «Dynamo: Amazon’s Highly Available Key-Value Store,» en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles*, ACM, 2007, págs. 205-220. DOI: 10.1145/1294261.1294281.
- [16] J. C. Corbett et al., «Spanner: Google’s Globally Distributed Database,» *ACM Transactions on Computer Systems*, vol. 31, n.º 3, págs. 1-22, 2013. DOI: 10.1145/2491245.
- [17] T. Taipalus, «Database Management System Performance Comparisons: A Systematic Literature Review,» *Journal of Systems and Software*, vol. 208, págs. 111872, 2024. DOI: 10.1016/j.jss.2023.111872.
- [18] E. Pina, F. Sá y J. Bernardino, «NewSQL Databases Assessment: CockroachDB, MariaDB Xpand, and VoltDB,» *Future Internet*, vol. 15, n.º 1, págs. 10, 2023. DOI: 10.3390/fi15010010.